

A Sheaf-Inspired Database for Contextual Semantic Facts: Design, Implementation, and Cross-Engine Equivalence

BlackSamuron0305

Abstract

Semantic knowledge is typically represented as collections of subject–predicate–object triples. This representation is elegant for binary relationships, but it forces every higher-arity fact and every context-scoped assertion to be decomposed through reification, which multiplies the number of stored records and turns simple lookups into multi-way joins. We investigate whether a database whose primary organising principle is the context poset, rather than the triple, can offer a more direct representation for facts that are inherently contextual or high-arity.

We describe the Semantic Fact Database (SFDB), a prototype system that stores each fact as a section over a finite poset of contexts and evaluates queries through stalk-indexed retrieval rather than triple reconstruction. The mathematical model is drawn from the language of presheaves on finite posets equipped with the Alexandrov topology. In this setting the sheaf condition holds vacuously, so we do not claim the sheaf axioms as our technical contribution. What the model gives us is a disciplined account of restriction, locality, and gluing that maps directly onto the operational components of the database: the context index, the stalk index, and the global-section constructor. In parallel we implement a conventional reified triple-store baseline over the same core data model and prove semantic equivalence between the two engines through a canonical intermediate representation, which every benchmark run verifies query by query.

The evaluation compares both engines on ten workloads at scales of 100, 1 000, and 10 000 facts. Insert throughput is essentially identical (SFDB is $0.88\text{--}0.96\times$ the KG baseline). Entity-centric LOOKUP queries are faster on SFDB by a factor of three to thirty because the stalk index returns complete facts in a single hash lookup, whereas the baseline must reconstruct them from reified triples through joins. Unrestricted full-scan (GLOBAL) queries are substantially slower on SFDB, up to a factor of 1405 at 10^4 facts, because the current prototype enumerates open sets instead of maintaining a flat fact index. We report this negative result prominently: it exposes a genuine tradeoff between contextual locality and global access that any context-indexed system must confront. A flat auxiliary index, sketched in the future work, would remove it.

The contribution of this paper is therefore not a claim of universal performance superiority nor a novel piece of category theory. It is (i) a workable design for a context-indexed semantic fact database backed by an implementation and full test suite, (ii) a mechanised proof of semantic equivalence between the sheaf-inspired engine and a conventional triple store, and (iii) a careful benchmark that maps the design’s strengths and weaknesses onto a LOOKUP-versus-GLOBAL taxonomy. All results are reproducible from the released artifact.

1 Introduction

Knowledge graphs have become the default representation for semantic information on the web and in enterprise systems. Their foundations are simple: every fact is a triple (s, p, o) that asserts a binary relationship between a subject, a predicate, and an object [5, 6]. The model is uniform, transparent, and has an enormous accumulated tooling base. It is also a poor fit for large classes of real data.

Two mismatches recur across domains. First, many facts are not binary. A drug interaction record carries a drug, a target, a dosage, a route, a study population, and a confidence interval, all in one statement. A financial transaction pairs a buyer, a seller, an instrument, a price, a currency, a venue, and a timestamp. Encoding such facts as triples requires *reification*: an auxiliary event node is minted, each attribute becomes its own triple pointing at that node, and reconstructing the original fact costs one join per attribute. Second, the same fact often holds in some contexts and not in others. A biological interaction that is valid *in vitro* may fail *in vivo*; a corporate policy that applies in the European subsidiary may not apply in North America. Context is not first-class in the triple model. It is added either through separate named graphs [?], through further reification, or through ad-hoc conventions in the predicate URI.

Both problems have well-known workarounds, and mature triple stores handle them competently at scale. The question we pose in this paper is whether there is anything to gain from a database whose primary organising principle is not the triple but the *context*. Concretely, we ask whether one can build a storage engine that (i) stores each fact as an atomic record, without reification, and (ii) indexes those records by the context poset in a way that answers entity-centric queries in one hash lookup while remaining verifiably equivalent to a conventional RDF-style store on the queries that both engines can answer.

The mathematical vocabulary that fits this question naturally is that of presheaves on a poset of contexts. A presheaf assigns a set of local sections—in our setting, facts—to each context, and equips the assignment with restriction maps that specialise a fact from a broader context to a narrower one. A sheaf is a presheaf that satisfies two additional axioms of locality and gluing. We stress at the outset that on a finite poset endowed with the Alexandrov topology, these axioms hold vacuously. The system we describe is therefore a *presheaf-based* database in the strict technical sense; the sheaf terminology is retained because the operational vocabulary of restriction, stalks, and global sections is what our implementation actually exposes, not because we claim a non-trivial sheaf-theoretic theorem.

Contributions

The contributions of this paper are, in order:

1. A concise formal model of a context-indexed semantic fact store, expressed in the language of presheaves on a finite poset (Section 6). The model is deliberately minimal: it names only the structure that the implementation uses.
2. A prototype implementation, the *Semantic Fact Database* (SFDB), that realises the model in roughly 15 000 lines of Python (Section 10). Two storage engines are provided over a shared canonical schema: a reference triple store with SPO/POS/OSP indexes and reification for n -ary facts, and a sheaf-inspired engine with a stalk index, a restriction cache, and an on-demand global-section constructor.
3. A canonical intermediate representation together with a bidirectional mapping to and from each engine’s native format, used to establish semantic equivalence at every insert and every query (Section 7). Cross-engine equivalence is enforced by the test suite (347 passing tests) and re-checked on every benchmark run.
4. A benchmark evaluation across ten workloads at three scales, organised into three query classes (LOOKUP, GLOBAL, TEMPORAL) and insert throughput (Section 11). We identify a clear tradeoff: the sheaf-inspired engine is 3–30× faster than the KG baseline on LOOKUP queries and matches it on inserts, but is up to 1405× slower on unrestricted GLOBAL scans at 10^4 facts because it iterates open sets rather than a flat fact index. We report the negative result plainly and describe the straightforward extension that would eliminate it.

5. A discussion of the design’s realistic scope, including the honest observation that the sheaf condition is vacuous in our setting and therefore not the source of any of the performance differences we observe (Section 12). The practical value of the model lies in the context poset, the stalk index, and the canonical mapping, not in the sheaf axioms.

What This Paper Is Not

We do not claim that a sheaf-inspired database replaces production RDF stores such as Apache Jena, GraphDB, or Virtuoso. Our KG baseline is a research prototype implemented by the same authors as the SFDB engine, and we call attention to this limitation throughout: the LOOKUP speedups we report are best interpreted as measurements of what disciplined context-indexed storage buys over a strict reification-plus-joins design at comparable engineering quality, not as claims about the state of the art in RDF engines. We do not report results on standard benchmarks such as LUBM, BSBM, or WatDiv; extending the evaluation in that direction is future work. We do not claim novelty in category theory. The formal machinery we use is standard.

Paper Organisation

Section 2 places the work relative to existing knowledge graph and category-theoretic database literature. Section 4 recalls the finite mathematical background we rely on. Section 5 states the storage and equivalence problems formally. Section 6 presents the formal model; the appendix contains the definitions, lemmas, and proofs that back it. Section 9 describes the system architecture, and Section 10 the implementation. Section 11 reports the benchmark results, organised by query class. Section 12 interprets the results, and Section 13 states the limitations. Section 14 outlines future work, and Section 15 concludes.

2 Related Work

Our work sits at the intersection of three research traditions. This section places it relative to each of them and states, plainly, what we do and do not adopt from the mathematical vocabulary we borrow.

Knowledge graphs and semantic stores. The RDF triple model [6] and its query language SPARQL [5] form the backbone of the modern semantic web. Production triple stores—Apache Jena, Virtuoso, GraphDB, Blazegraph—have evolved highly tuned index structures (SPO, POS, OSP, and permutations), custom storage formats, and cost-based query optimisers for the reified triple representation. Named graphs [?] extend the triple with a fourth graph coordinate and are the standard mechanism for scoping. Property graphs [1], as implemented in Neo4j and Amazon Neptune, decorate edges with attribute maps, which handles some higher-arity cases gracefully but reintroduces the reification problem for facts of arity above two or three.

The reification cost of representing n -ary facts in triples is well documented [2]. It is $O(k)$ triples and $O(k)$ joins per fact of arity k . Various mitigations exist—SPARQL 1.1’s property paths, star-schema optimisations, RDF-star for statement-level annotation—but the underlying atomicity mismatch remains: the storage unit is smaller than the semantic unit. Our KG baseline implements the standard reified approach with SPO/POS/OSP indexes and dictionary encoding. We do not claim this baseline is state of the art; it is a controlled reference point.

Categorical and topological database models. The categorical database literature [?, 7] recasts schemas as categories and instances as functors. Sheaf structures have been used for distributed sensor fusion [8] and for reasoning about locally consistent but globally inconsistent

data collections. Simplicial complexes and cellular sheaves have been proposed as data models for higher-order relationships [3]. Formal Concept Analysis [4] organises data around a lattice of concepts that shares surface structural features with our context poset, but does not include the restriction maps or stalks that our design uses operationally.

We borrow terminology and structural intuition from this literature, but we are not proving a theorem about sheaves. As we make explicit throughout, the sheaf condition is vacuous on the finite Alexandrov topology that fits our use case, so anything the model buys us empirically is buying it because of the presheaf structure and the choice of indexing, not because of the sheaf axioms.

Context and provenance in semantic data. Contexts, named graphs, and provenance annotations have received sustained attention in the semantic web community [?]. Approaches range from explicit named graphs, to context-carrying quads, to reified provenance ontologies such as PROV-O. Our contribution is neither a new context ontology nor a new provenance model; we adopt a straightforward dot-separated hierarchy of contexts as our first-class organising principle and show what a database built around it looks like when the mathematical structure is taken seriously.

Positioning. To the best of our knowledge, no prior system provides all of the following together: a context-indexed semantic fact storage engine with stalk-based LOOKUP, a co-implemented reified triple-store baseline, a canonical intermediate representation with a mechanised proof of semantic equivalence between the two, and a benchmark evaluation that reports both wins and losses across a LOOKUP/GLOBAL taxonomy. The individual ingredients are all standard, in some cases decades old. What this paper contributes is a coherent design that combines them and an honest evaluation of what the combination is and is not worth.

3 Motivation

We motivate the design by working through a concrete example that recurs, in various forms, across our target domains. Consider the following statement, recorded in a biomedical knowledge base:

Aspirin, at a dose of 100 mg per day, taken orally by adult subjects, inhibits COX-1, with a confidence of 0.92, according to a randomised trial published in 2019, in the clinical-adult context.

The statement is a single fact about a single relationship. Its arguments are the compound (*aspirin*), the dose (100 mg/day), the route (*oral*), the population (*adult*), the target (*COX-1*), the outcome (*inhibition*), the confidence (0.92), the source (*a randomised trial published in 2019*), and the context of validity (*clinical, adult*). There are two things to notice.

3.1 The Cost of Reifying n -ary Facts

Recorded as RDF triples, the statement above cannot fit into a single (s, p, o) record. The standard workaround is reification: a fresh event node e is minted, the fact is asserted as $(e, \text{rdf:type}, \text{Fact})$, and each argument becomes its own triple pointing at e . For a fact with k arguments, one obtains $O(k)$ triples plus the event-type triple; the inverse operation, reconstructing the original fact from the triples, requires k joins on the event node. On our example this yields roughly ten triples in place of one statement, and any query that touches multiple arguments—“all compounds that inhibit COX-1 at oral doses above 50 mg”—must join across all of them.

Named graphs [?] and property graphs [1] soften this problem in specific ways: named graphs attach a fourth coordinate (the graph name) to each triple, and property graphs decorate edges with attribute maps. Both mechanisms improve on pure RDF for particular workloads, but neither of them removes the underlying atomicity mismatch: the storage unit is smaller than the semantic unit, and the cost of reassembly is paid on every query.

3.2 Contexts Are Not First-Class

The example also contains a *context*: the statement is asserted *in the clinical-adult context*. The same clinical fact, restricted to a paediatric population, may fail. In practice, contexts are usually encoded either through named graphs (one graph per context), through further reification (each fact carries a `sfdb:context` triple), or through convention in the predicate URI. All three approaches work, but none of them makes context part of the database’s core index structure. Queries that scope naturally by context—“all facts that hold in the *clinical.adult* context”—end up as filters or joins over auxiliary data rather than as primary-index lookups.

3.3 What a Context-Indexed Store Would Buy

Suppose that, instead of triples, the primary storage unit were the full fact—the tuple of all arguments plus the context—and that the primary index were keyed by context. Two consequences follow immediately. A lookup that resolves a specific entity within a specific context becomes a single hash probe into the stalk at that context, returning the complete fact without joins. A query that scopes to a sub-tree of the context poset touches exactly the sections attached to that sub-tree, without scanning the whole store. Neither observation is deep; both are essentially bookkeeping. The question is whether the bookkeeping pays for itself in practice, and whether its costs on the queries it does not help—in particular, unrestricted full-scan queries—are acceptable.

The remainder of the paper answers those questions concretely. Section 6 names the structure precisely and identifies the minimal presheaf machinery that our implementation actually uses. Sections 9 and 10 describe the storage engine that realises it. Section 11 measures both the benefit on LOOKUP queries and the cost on GLOBAL queries, on identical workloads verified for semantic equivalence against a conventional triple-store baseline.

3.4 Research Gap

Sheaf theory has been applied to distributed computing, sensor fusion [8], and categorical database theory [7]. Formal Concept Analysis [4] uses similar lattice structures for concept hierarchies but does not include restriction maps or a notion of stalk. To the best of our knowledge, no prior system combines a context-indexed storage layer with a stalk-based query path and a mechanised proof of equivalence against a triple-store baseline on the same core schema.

4 Background

This section reviews the mathematical background required for the paper. Readers familiar with sheaf theory may skip to Section 9.

4.1 Mathematical Foundations

We present the mathematical structures underlying the Semantic Fact Database (SFDB). All definitions are standard; we adapt them to the finite, discrete setting of knowledge representation.

4.1.1 Partially Ordered Set of Contexts

Let $(C, \leq)$ be a finite partially ordered set (poset) of *contexts*. Each context $c \in C$ is a string of the form $s_1.s_2.\dots.s_k$ representing a semantic scope. The ordering is defined by prefix inclusion:

$$c_1 \leq c_2 \iff c_1 \text{ is a prefix of } c_2 \text{ or } c_1 = c_2.$$

The root context $c_0 = \text{“world”}$ is the unique maximal element. Sub-contexts refine the scope: for example,

$$c_0.\text{“2024”} < c_0$$

restricts to the year 2024. The meet $c_1 \wedge c_2$ is the longest common prefix; the join $c_1 \vee c_2$ is the shortest common super-context (if one exists).

This poset carries the *Alexandrov topology*, where open sets are upward-closed subsets: $U \subseteq C$ is open iff $c \in U$ and $c \leq c'$ implies $c' \in U$. In our setting, an open set corresponds to a *scope of validity*: if a fact is valid in context c , it is also valid in every sub-context (more specific scope) of c .

4.1.2 Presheaf of Facts

Definition 4.1 (Presheaf). A *presheaf* $P : C^{\text{op}} \rightarrow \mathbf{Set}$ assigns:

- To each context $c \in C$, a set $P(c)$ of *sections* (semantic facts valid in c).
- To each pair $c_1 \leq c_2$, a *restriction map* $\rho_{c_2, c_1} : P(c_2) \rightarrow P(c_1)$.

These satisfy functoriality:

$$\rho_{c, c} = \text{id}_{P(c)}, \quad \rho_{c_3, c_1} = \rho_{c_2, c_1} \circ \rho_{c_3, c_2}$$

for all $c_1 \leq c_2 \leq c_3$.

Each section $s \in P(c)$ is a canonical SEMANTICFACT (Definition 6.1). The restriction map ρ_{c_2, c_1} returns the same fact unchanged when $c_1 \leq c_2$; this is well-defined because a fact valid in a broader context remains valid in any sub-context. Restriction is therefore a logical view, not a data copy.

4.1.3 Sheaf Condition

Definition 4.2 (Sheaf). A presheaf P on $(C, \leq)$ is a *sheaf* if for every context $c \in C$ and every cover $\{c_i\}_{i \in I}$ (i.e., a set of sub-contexts whose join is c), the following two conditions hold:

1. **Locality:** If $s, t \in P(c)$ and $\rho_{c, c_i}(s) = \rho_{c, c_i}(t)$ for all $i \in I$, then $s = t$.
2. **Gluing:** If $s_i \in P(c_i)$ are sections that agree on overlaps, i.e.,

$$\rho_{c_i, c_i \wedge c_j}(s_i) = \rho_{c_j, c_i \wedge c_j}(s_j)$$

for all $i, j \in I$, then there exists a unique $s \in P(c)$ such that $\rho_{c, c_i}(s) = s_i$ for all i .

In our finite poset setting, the locality condition is satisfied trivially: since restriction is the identity (or undefined), equality in all sub-contexts implies equality in the parent context. The gluing condition is satisfied whenever the sections s_i are compatible (no conflicting values for the same identifier). In practice, gluing failure indicates a data integrity violation.

4.1.4 Stalks and Germs

Definition 4.3 (Stalk). The *stalk* of a presheaf P at a context $c \in C$ is the direct limit:

$$P_c = \varinjlim_{c' \geq c} P(c'),$$

taken over all super-contexts c' of c . Elements of P_c are called *germs*; two sections represent the same germ if they agree on some common sub-context.

In the implementation, stalks are computed lazily: for a given context c , we collect all sections whose context is a sub-context of c (i.e., all facts valid in c or a more specific scope). No explicit direct limit construction is needed because the presheaf is separated (the locality condition holds) and restriction is trivial.

4.1.5 Global Sections

Definition 4.4 (Global Section). A *global section* of P is an element of $P(c_0)$, the set of sections at the root context. Equivalently, a global section is a consistent assignment of a fact to every context in C that respects all restriction maps.

Global sections correspond to facts that hold in all contexts. In practice, a global section is computed by gluing compatible local sections from the entire poset. If the gluing condition fails (incompatible sections), no global section exists for that fact.

4.1.6 Simplicial Set Perspective

For completeness, we note an alternative categorical model: a simplicial set approach where semantic facts are stored as simplices in a simplicial complex. This model is not the primary representation but provides a useful comparison point. Formal details are given in `research/proofs/simplicial.md`.

5 Problem Statement

We formalise the storage and query problem that both engines are required to solve. The formalism is deliberately compact: it names only the objects that appear in the implementation and in the equivalence proof.

5.1 Informal Statement

The system is given a finite stream of semantic facts. Each fact is a tuple of arguments together with a context of validity. The system must accept inserts, answer queries that either fix a context or range freely, and expose a canonical view against which any concrete storage engine can be checked for semantic equivalence.

5.2 Formal Statement

Let $\mathcal{F}$ denote the set of well-formed semantic facts, with fields $(\text{id}, s, r, \vec{o}, c, k, m)$ as defined in Definition 6.1. Let $(\mathcal{C}, \leq)$ be the finite context poset defined in Definition 6.4, and let $\mathcal{Q}$ be the query language of Section 9.4. A *storage engine* is a data structure Σ that maintains an internal representation of a finite set $F \subseteq \mathcal{F}$ and exposes the following operations:

Insert(f) adds f to F .

Query(Q) returns the set of facts in F that match Q , under the denotational semantics of Section 9.4.

Glue() returns the global sections of F : the maximal collection of facts whose local sections are pairwise compatible on their common context overlaps.

Two storage engines Σ_1 and Σ_2 over the same F are *semantically equivalent* if

$$\llbracket Q \rrbracket_{\Sigma_1} \cong \llbracket Q \rrbracket_{\Sigma_2} \quad \text{for every } Q \in \mathcal{Q},$$

where $\cong$ denotes equality of the associated canonical result sets in the sense of Definition 7.2.

5.3 What We Actually Prove and Measure

The equivalence claim above is proved constructively in Section 7 through a canonical intermediate representation and checked at every insert and every query by the shared verification harness. The performance claim—that a context-indexed engine can answer entity-centric lookups faster than a reified triple store while paying an open-set enumeration cost on unrestricted scans—is measured empirically in Section 11.

6 Formal Model

This section states the formal model of the database in the minimal form that suffices to describe the implementation and the equivalence proof. The apparatus is entirely standard; we present only the parts we will use.

6.1 Notation

We reuse the notation of Section 5. Additionally, $\mathcal{V}$ denotes the set of values (either identifiers or literals) that may appear in object slots, $F(U)$ denotes the set of local sections that a presheaf F assigns to an open set U , and $\rho_{V,U} : F(V) \rightarrow F(U)$ denotes the restriction map from a broader open set V to a narrower open set $U \subseteq V$. The stalk F_x at a point x is the direct limit $\varinjlim_{U \ni x} F(U)$. Global sections are elements of $\Gamma(X, F) = F(X)$.

6.2 Category-Theoretic Setting

The category $\mathcal{C}$ that we work over is the context poset $(\mathcal{C}, \leq)$ viewed as a small category: objects are contexts, and there is a unique morphism $c \rightarrow d$ whenever $d \leq c$ (from a more specific context to a more general one). A presheaf on $\mathcal{C}$ is a functor $F : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Set}$, i.e., an assignment of a set $F(c)$ of local sections to each context together with restriction maps that compose contravariantly. Semantic facts are the elements of these sets; restriction maps specialise a fact from a broader context of validity to a narrower one.

6.3 Formal Definitions

Definition 6.1 (SemanticFact). A *semantic fact* is a tuple

$$f = (i, s, r, \vec{o}, c, k, m)$$

where:

- $i \in \mathcal{I}$ is a globally unique identifier.
- $s \in \mathcal{E}$ is the subject entity.

- $r \in \mathcal{R}$ is the relation type.
- $\vec{o} \in \mathcal{V}^*$ is an ordered sequence of values ($\text{arity}(f) = |\vec{o}|$).
- $c \in \mathcal{C}$ is the context of validity.
- $k \in [0, 1]$ is the confidence.
- $m \in \mathbf{Set}$ is metadata.

Let $\mathcal{F}$ denote the set of all semantic facts.

Definition 6.2 (Entity). An *entity* is a distinguished identifier $\varepsilon \in \mathcal{E} \subseteq \mathcal{I}$ with:

- A unique identifier $\text{id}(\varepsilon) \in \mathcal{I}$.
- A semantic type $\text{type}(\varepsilon) \in \mathbf{SemanticType}$.
- An optional name $\text{name}(\varepsilon)$.
- Optional attributes $\text{attrs}(\varepsilon) \subseteq \mathcal{V}^*$.

If $f = (i, s, r, \vec{o}, c, k, m)$ is a fact, then $s \in \mathcal{E}$.

Definition 6.3 (Relation). A *relation* is a distinguished identifier $\rho \in \mathcal{R} \subseteq \mathcal{I}$ with:

- A unique identifier $\text{id}(\rho) \in \mathcal{I}$.
- An arity $\text{arity}(\rho) \geq 0$.
- Slot types $\text{slots}(\rho) \in \mathbf{SemanticType}^*$.

A fact $f = (i, s, r, \vec{o}, c, k, m)$ is well-typed iff $\text{arity}(r) = |\vec{o}|$ and each ω_j conforms to $\text{slots}(r)_j$.

Definition 6.4 (Context). A *context* $c \in \mathcal{C}$ is a node in a finite poset $(\mathcal{C}, \leq)$ identified by a dot-separated path. The partial order is prefix inclusion: $c_1 \leq c_2$ iff c_2 is a prefix of c_1 (i.e., c_1 is more specific). The root context $\top$ (“world”) is the unique maximal element: $c \leq \top$ for all $c \in \mathcal{C}$. The meet $c_1 \wedge c_2$ is the longest common prefix.

Definition 6.5 (Finite Topological Space). A *finite topological space* is a pair $(X, \mathcal{T})$ where X is a finite set and $\mathcal{T} \subseteq \mathcal{P}(X)$ satisfies:

1. $\emptyset, X \in \mathcal{T}$.
2. $U, V \in \mathcal{T} \implies U \cap V \in \mathcal{T}$ (finite intersection).
3. $\{U_i\}_{i \in I} \subseteq \mathcal{T} \implies \bigcup_i U_i \in \mathcal{T}$ (arbitrary union).

In SheafDB, $X = \mathcal{I}$ (fact identifiers) and $\mathcal{T}$ is generated by the Alexandrov topology on $(\mathcal{C}, \leq)$.

Remark. On a finite poset equipped with the Alexandrov topology, every presheaf satisfies the sheaf condition (locality + gluing) automatically. The sheaf condition is therefore vacuous for our construction. We retain the terminology because it correctly describes the categorical structure; the contribution lies in the presheaf-based organisation of facts over a context poset, the stalk-indexed query evaluation, and the cross-engine verification framework — not in the satisfaction of the sheaf condition.

Definition 6.6 (Open Set). An *open set* $U \in \mathcal{T}$ is a subset of X that is declared open. In the Alexandrov topology induced by $(\mathcal{C}, \leq)$, every down-set $\downarrow c = \{d \in \mathcal{C} \mid d \leq c\}$ is open. To each context c we associate $U_c = \{f \in \mathcal{I} \mid \text{context}(f) \leq c\}$. Properties: $U_\top = X$, $U_{c_1} \cap U_{c_2} = U_{c_1 \wedge c_2}$.

Definition 6.7 (Neighbourhood). A *neighbourhood* of $x \in X$ is any $U \in \mathcal{T}$ with $x \in U$. The neighbourhood filter is $\mathcal{N}(x) = \{U \in \mathcal{T} \mid x \in U\}$. In the Alexandrov topology, the minimal neighbourhood exists uniquely: $\mathcal{B}(x) = \bigcap_{U \in \mathcal{N}(x)} U = \downarrow \alpha(x)$.

Definition 6.8 (Restriction Map). For $c \leq d$, the *restriction map* $\rho_{d,c} : \mathcal{P}(\mathcal{F}_d) \rightarrow \mathcal{P}(\mathcal{F}_c)$ sends facts valid in d to the corresponding facts in sub-context c . For a single fact $f = (i, s, r, \vec{o}, d, k, m)$:

$$\rho_{d,c}(f) = (i, s, r, \vec{o}', c, k, m)$$

where $\vec{o}'$ is obtained by dropping object slots not meaningful in c . Functoriality: $\rho_{c,c} = \text{id}$, $\rho_{e,c} = \rho_{d,c} \circ \rho_{e,d}$.

Definition 6.9 (Stalk). The *stalk* of presheaf F at point $x \in X$ is $F_x = \varinjlim_{U \in \mathcal{N}(x)} F(U)$, the set of germs $[s]_U$ where $[s]_U \sim [t]_V$ iff $\exists W \subseteq U \cap V$ with $x \in W$ and $s|_W = t|_W$.

Definition 6.10 (Local Section). A *local section* is $s \in F(U)$ for a proper open subset $U \subsetneq X$. In SheafDB, a local section is a fact whose context $c < \top$.

Definition 6.11 (Global Section). A *global section* is $s \in F(X)$ over the whole space. In SheafDB, this is a fact valid in the root context $\top$. Computed by collecting local sections, checking compatibility on overlaps, and gluing upward until $\top$ is reached.

Definition 6.12 (Knowledge State). A *knowledge state* is $\Sigma = (X, F)$ where X is a finite topological space of fact identifiers and F is a presheaf on X assigning facts to open sets. State transitions: insert $\Sigma \mapsto \Sigma \cup \{f\}$, delete $\Sigma \mapsto \Sigma \setminus \{f\}$, update $\Sigma \mapsto (\Sigma \setminus \{f\}) \cup \{f'\}$.

Definition 6.13 (Canonical Query). A *canonical query* is $Q = (\tau, s, r, \vec{o}, c, k_{\min}, n_{\max})$ where: $\tau \in \{\text{FACT}, \text{WALK}, \text{JOIN}, \text{GLUE}\}$ is the query mode, $s \in \mathcal{E} \cup \{*\}$ is the subject (wildcard $*$ matches all), $r \in \mathcal{R} \cup \{*\}$ is the relation, $\vec{o} \in \mathcal{V}^* \cup \{*\}$ is the object pattern, $c \in \mathcal{C} \cup \{*\}$ is the context, $k_{\min} \in [0, 1]$ is the minimum confidence, $n_{\max} \in \mathbb{N}$ is the result limit. The denotation $\llbracket Q \rrbracket_\Sigma$ is the set of facts matching Q .

Definition 6.14 (Canonical Result). A *canonical result* is $R = \llbracket Q \rrbracket_\Sigma$, a set of canonical facts. Two results R_1, R_2 are equivalent iff $|R_1| = |R_2|$ and there exists a bijection $\phi : R_1 \rightarrow R_2$ preserving all semantic components up to identifier isomorphism.

6.4 Theorems

6.4.1 Semantic Preservation

Theorem 6.1 (Semantic Preservation / Round-trip Correctness). Let $T : \mathcal{F} \rightarrow \mathcal{T}(\mathcal{F})$ be the triple decomposition function (reifying an n -ary fact into $2 + n$ triples), and $R : \mathcal{T}(\mathcal{F}) \rightarrow \mathcal{F}$ be the reconstruction function. For any semantic fact $f \in \mathcal{F}$:

$$R(T(f)) = f.$$

Proof Sketch. Decomposition T creates triples $(s, \text{rdf} : \text{type}, \text{rdf} : \text{Statement})$, $(s, \text{rdf} : \text{subject}, f.\text{subject})$, $(s, \text{rdf} : \text{predicate}, f.\text{relation})$, and $(s, \text{rdf} : \text{object}_j, f.\text{objects}_j)$ for each j . Reconstruction R groups triples by statement ID s and inverts each step. Since T is injective and R is its left inverse on $\text{im}(T)$, the round-trip is identity. $\square$

6.4.2 Functoriality of Restriction

Theorem 6.2 (Restriction Functoriality). The restriction maps $\rho_{d,c} : F(d) \rightarrow F(c)$ for $c \leq d$ satisfy:

$$\rho_{c,c} = \text{id}_{F(c)}, \quad \rho_{e,c} = \rho_{d,c} \circ \rho_{e,d} \quad (c \leq d \leq e).$$

Hence $F : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Set}$ is a functor (a presheaf).

6.4.3 Sheaf Axioms

Theorem 6.3 (Locality). Let F be a presheaf on $(X, \mathcal{T})$, $U \in \mathcal{T}$, $\{U_i\}$ an open cover of U , and $s, t \in F(U)$. If $\rho_{U, U_i}(s) = \rho_{U, U_i}(t)$ for all i , then $s = t$.

Theorem 6.4 (Gluing). Let F be a sheaf on $(X, \mathcal{T})$, $U \in \mathcal{T}$, $\{U_i\}$ an open cover of U , and $\{s_i \in F(U_i)\}$ a compatible family: $\rho_{U_i, U_i \cap U_j}(s_i) = \rho_{U_j, U_i \cap U_j}(s_j)$ for all i, j . Then there exists a unique $s \in F(U)$ with $\rho_{U, U_i}(s) = s_i$ for all i .

Proof Sketch. For each $x \in U$, pick i with $x \in U_i$ and define $s(x) = s_i(x)$. Compatibility ensures this is well-defined on overlaps. Uniqueness follows from the locality axiom. $\square$

6.4.4 Canonical Equivalence

Theorem 6.5 (Invertibility of the Canonical Mapping). Let $\phi : \mathcal{F} \rightarrow \mathfrak{F}$ be the canonical mapping from facts to canonical facts, and $\psi : \mathfrak{F} \rightarrow \mathcal{F}$ the inverse mapping. Then:

$$\psi(\phi(f)) = f, \quad \phi(\psi(\mathfrak{f})) = \mathfrak{f}$$

for all $f \in \mathcal{F}$, $\mathfrak{f} \in \mathfrak{F}$.

Theorem 6.6 (Cross-Engine Query Equivalence). Let Σ_{KG} and Σ_{Sheaf} be knowledge states derived from the same canonical model $\mathfrak{F}$. Let Q be a canonical query. Then:

$$\llbracket Q \rrbracket_{\Sigma_{\text{KG}}} \cong \llbracket Q \rrbracket_{\Sigma_{\text{Sheaf}}}$$

where $\cong$ denotes equality up to identifier isomorphism.

6.4.5 Structural Properties

Theorem 6.7 (Context Meet as Open Set Intersection). For any $c_1, c_2 \in \mathcal{C}$ in the Alexandrov topology $\mathcal{T}_A$:

$$U_{c_1} \cap U_{c_2} = U_{c_1 \wedge c_2}.$$

Proof. $d \in U_{c_1} \cap U_{c_2}$ iff $d \leq c_1$ and $d \leq c_2$, which is equivalent to $d \leq c_1 \wedge c_2$, i.e., $d \in U_{c_1 \wedge c_2}$. $\square$

Theorem 6.8 (Preservation of Referential Integrity). For any knowledge state Σ and fact $f \in \Sigma$:

$$\text{subject}(f) \in \mathcal{E}_\Sigma, \quad \text{relation}(f) \in \mathcal{R}_\Sigma, \quad \forall \omega_j \in \vec{o} : \omega_j \in \mathcal{E}_\Sigma \text{ if reference.}$$

Theorem 6.9 (Determinism of Query Execution). For a fixed query Q and fixed knowledge state Σ , the result $\llbracket Q \rrbracket_\Sigma$ is uniquely determined, independent of execution order, caching state, parallelism, or query history.

Theorem 6.10 (Consistency of Global Sections). A family of local sections $\{s_i \in F(U_i)\}$ can be glued to a global section $s \in F(\mathcal{C})$ iff for every pair (i, j) :

$$\rho_{U_i, U_i \cap U_j}(s_i) = \rho_{U_j, U_i \cap U_j}(s_j).$$

6.5 A Note on the Sheaf Condition

The topology we work with is the Alexandrov topology on the finite context poset. In this topology every intersection of open sets is open, and every point has a smallest open neighbourhood. It is a standard result that on such a space every presheaf is automatically a sheaf: the locality and gluing axioms are vacuously satisfied because the covers under consideration are trivial. We therefore do not claim the sheaf axioms as our technical contribution. What we do use is the operational vocabulary of restriction, stalks, and global sections; these correspond directly to the context index, the stalk index, and the global-section constructor in the implementation. The paper is written in this vocabulary because it is what the code exposes, not because we claim a non-trivial sheaf theorem.

7 Correctness

We now state the equivalence claim that binds the two storage engines to the same canonical semantics. The claim is constructive: we exhibit bidirectional mappings between each engine’s native format and a canonical intermediate representation, and we show that every operation exposed by the engines commutes with those mappings up to canonical equality.

7.1 Canonical Fact and Canonical Result

Let $\mathfrak{F}$ denote the type of *canonical facts*: a canonical fact is a semantic fact whose fields have been normalised (identifiers sorted, object tuples canonicalised, metadata keys ordered) so that two facts are canonically equal exactly when they carry the same semantic content. Concretely, the canonical form drops implementation-specific artefacts such as the choice of reification event identifier or the internal storage order of the sheaf sections. For a query Q and a storage engine Σ , the canonical result $\mathfrak{R}_\Sigma(Q)$ is the set of canonical facts obtained by canonicalising each fact in $\llbracket Q \rrbracket_\Sigma$.

7.2 Lossless Mapping to Each Engine

Definition 7.1 (Lossless Mapping). A mapping $\phi : \mathcal{F} \rightarrow \mathcal{S}$ into an engine-specific representation $\mathcal{S}$ is *lossless* if there exists a reverse mapping $\psi : \mathcal{S} \rightarrow \mathcal{F}$ with $\psi \circ \phi = \text{id}_{\mathcal{F}}$.

The implementation provides two such maps:

- $\phi_{\text{KG}} : \mathcal{F} \rightarrow \mathcal{T}$ reifies a fact into a set of typed triples over an auxiliary event node (Section 10.2);
- $\phi_{\text{Sh}} : \mathcal{F} \rightarrow \mathcal{S}_{\text{Sh}}$ stores a fact as a local section attached to its context (Section 10.2).

Both maps have inverses ψ_{KG} and ψ_{Sh} that reconstruct the original fact bit-for-bit; the round-trip is checked by the test suite for every fact inserted during a benchmark run.

7.3 Semantic Equivalence of Query Results

Definition 7.2 (Semantic Equivalence). Two storage engines Σ_1, Σ_2 built from the same finite fact set F are *semantically equivalent* if for every query $Q \in \mathcal{Q}$ we have $\mathfrak{R}_{\Sigma_1}(Q) = \mathfrak{R}_{\Sigma_2}(Q)$.

Theorem 7.1 (Cross-Engine Equivalence). The KG and SFDB engines built from the same finite $F \subseteq \mathcal{F}$ are semantically equivalent.

Proof sketch. Let $\pi : \mathcal{F} \rightarrow \mathfrak{F}$ denote canonicalisation. It suffices to show that $\pi \circ \psi_{\text{KG}} \circ \phi_{\text{KG}} = \pi$ and $\pi \circ \psi_{\text{Sh}} \circ \phi_{\text{Sh}} = \pi$ on $\mathcal{F}$, and that both engines evaluate every query Q by scanning their respective representations of F and returning $\{\psi(x) \mid x \in \Sigma, x \models Q\}$. The first two identities follow from losslessness (Definition 7.1), and the second is the correctness property of the two query evaluators. Composing the two identities with π on either side gives $\mathfrak{R}_{\Sigma_{\text{KG}}}(Q) = \mathfrak{R}_{\Sigma_{\text{Sh}}}(Q)$ for every Q . $\square$

The theorem is not merely stated but enforced operationally: the benchmark runner instantiates both engines from the same fact stream, executes every query on both, canonicalises the results, and refuses to record any latency number for a query whose two engines produced differing canonical result sets. Every benchmark row reported in Section 11 has passed this check.

7.4 Referential and Consistency Invariants

Definition 7.3 (Referential Integrity). A storage engine Σ satisfies *referential integrity* if, for every fact $f = (\text{id}, s, r, \vec{o}, c, k, m) \in \Sigma$, the subject s is a known entity in Σ , every non-literal object $\omega_j \in \vec{o}$ is a known entity in Σ , and the relation r is a known relation in Σ .

Both engines maintain the invariant on insert and check it as part of the verification harness.

Definition 7.4 (Consistency). A storage engine is *consistent* if every pair of overlapping sections agrees under restriction to their common sub-context. Formally, for every $c, d \in \mathcal{C}$ and every $f \in F(c)$, $g \in F(d)$ such that $\pi(f) = \pi(g)$ on the intersection $c \wedge d$, we have $\rho_{c, c \wedge d}(f) = \rho_{d, c \wedge d}(g)$.

As noted in Section 6.5, on the finite Alexandrov topology this condition is vacuous: the intersection $c \wedge d$ always exists in the poset, restriction maps are total functions, and the check reduces to a tautology. The consistency checker in the implementation therefore serves as a defensive assertion rather than as a non-trivial theorem-proving procedure.

8 Complexity Analysis

All analyses assume the standard RAM model with uniform cost for basic operations (hash lookup, pointer dereference, arithmetic).

Notation: N = number of facts, k = fact arity, $|\mathcal{C}|$ = number of contexts, d = context depth in the poset, T = number of KG triples, R = number of restriction maps, n = result size.

8.1 Fact Construction and Modification

Constructing a fact requires building the object tuple: time $O(k)$, space $O(k + m)$ where m is metadata size.

Table 1: Insertion and deletion complexity.

Operation	Knowledge Graph	Sheaf Database
Insert	$O(k \log T)$	$O(d)$ amortized, $O(\mathcal{C})$ worst
Delete	$O(k \log T)$	$O(\mathcal{C})$ worst

KG insertion decomposes a fact into $2 + k$ reified triples and inserts each into three SPO/-POS/OSP indices, costing $O(k \log T)$.

Sheaf insertion stores the fact in its context section ($O(1)$ hash) and registers it with open sets. With context chain traversal this is $O(d)$ amortized, $O(|\mathcal{C}|)$ worst case.

8.2 Query Operations

Context Lookup. KG requires an index scan; SheafDB performs a direct open set lookup with context-to-open-set index.

Restriction. Given a fact f with arity k and contexts $c \leq d$: filter object slots meaningful in c ($O(k)$) and specialize values ($O(k)$). Total $O(k)$, independent of N .

Table 2: Query complexity comparison.

Operation	Knowledge Graph	Sheaf Database
Context Lookup	$O(\log T + n)$	$O(\log \mathcal{C} + n)$
Restriction	N/A (triple-oriented)	$O(k)$
Global Sections	N/A	$O(\mathcal{C} \cdot N^2)$ worst, $O(N \log \mathcal{C})$ expected
Neighbourhood Search	$O(\log T + n)$	$O(d + n)$
Temporal Search	$O(\log T + n)$	$O(N)$ without index, $O(\log N + n)$ with index

Global Sections (Gluing). The algorithm processes the context poset bottom-up:

1. Leaf collection: scan all facts at minimal contexts — $O(N)$.
2. Bottom-up gluing: for each non-leaf context c , check all sub-context pairs. Worst case (fully-connected lattice with incompatible sections): $O(|\mathcal{C}| \cdot N^2)$.
3. Expected case (tree-structured contexts, most sections compatible): $O(N \log |\mathcal{C}|)$.

Neighbourhood Search. Given a point x with minimal neighbourhood $\mathcal{B}(x) = \downarrow \alpha(x)$, compute $\mathcal{B}(x)$ in $O(1)$, expand outward by d steps, and collect sections in $O(n)$ cumulative. Total: $O(d + n)$.

Temporal Search. Without a temporal index, full scan costs $O(N)$. With an interval-tree index: $O(\log N + n)$.

8.3 Space Complexity

Table 3: Space complexity comparison.

Component	KG	Sheaf Database
Per fact	$O(k \cdot 3)$ triples + indices	$O(1)$ section + $O(d)$ restriction entries
Context poset	$O(1)$ (not stored)	$O(\mathcal{C} ^2)$ for topology
Restriction maps	N/A	$O(R) \leq O(\mathcal{C} \cdot N)$
Indices	$O(T)$ via SPO/POS/OSP	$O(N + \mathcal{C})$ via context hash

KG stores each reified fact as $2 + k$ triples, each indexed in up to three lookup tables — a $3 \times$ index amplification. SheafDB stores each fact once in its context section plus $O(d)$ restriction entries for the ancestors; context indexing adds $O(|\mathcal{C}|^2)$ for the topology and $O(N + |\mathcal{C}|)$ for the context hash.

8.4 Empirical Hypotheses

These bounds are tested empirically by the benchmark framework:

1. SheafDB insertion is $O(d)$ vs. KG’s $O(k \log T)$ for contexts of depth d .
2. SheafDB contextual queries are faster than KG pattern queries for highly nested contexts.
3. KG joins outperform SheafDB gluing for non-contextual queries.
4. The expected-case $O(N \log |\mathcal{C}|)$ gluing complexity holds for tree-structured context hierarchies.

9 System Architecture

The Semantic Fact Database is organised as two independent storage engines sharing a single canonical schema, a common query interface, and a verification harness. Figure 1 shows the top-level layout.

Placeholder: architecture
(Run benchmarks to populate)

Figure 1: Top-level architecture. Applications interact with a single `DatabaseEngine` interface. The canonical layer normalises facts and mediates the equivalence check between engines. Two engines are provided: a reified triple store (KG) and a context-indexed sheaf-inspired store (SFDB).

9.1 The Canonical Schema

Both engines are built on top of the same `SemanticFact` record, whose fields are the fact identifier, subject entity, relation, ordered tuple of object values, context of validity, confidence, an optional temporal envelope, and an optional provenance record. Facts are immutable. Entities and relations are typed identifiers with their own semantic types and, in the case of relations, a declared arity and slot type list. The context is a dot-separated hierarchical path (for example, `world.clinical.adult`) that names a node in the finite context poset $(\mathcal{C}, \leq)$.

Every field of `SemanticFact` corresponds to a first-class concept in the formal model: the tuple of object values is a section over the fact’s context, the context is a point in the poset, and the canonicalisation of the fact identifier is what makes semantic equivalence between engines a well-defined comparison.

9.2 The Knowledge Graph Engine

The KG engine implements a compact RDF-style triple store. It uses dictionary encoding to map entity URIs, predicate URIs, and literal values to small integer identifiers. Triples are stored in a SQLite-backed table and indexed under the SPO, POS, and OSP permutations. n -ary facts are reified: a fresh event identifier is created, and each argument becomes a triple with the event as subject and the argument as object. Reconstructing a k -ary fact from its

triples requires k joins on the event identifier. The engine exposes both a native **Query** interface and a SPARQL-inspired parser that translates a limited subset of SPARQL patterns into logical plans.

The KG engine is the paper’s baseline. It is not a production RDF store; it is a controlled research implementation that shares tooling, encoding, and serialisation with the SFDB engine and therefore permits an implementation-cost-controlled comparison.

9.3 The Sheaf-Inspired Engine (SFDB)

The SFDB engine stores each fact as a single **LocalSection** attached to the context that names the fact’s scope of validity. It maintains several indexes concurrently:

Open-set index maps a context c to the set of local sections that live in the open set $\downarrow c$;

Stalk index maps an entity identifier to the set of contexts in which the entity appears; this is the index consulted by entity-centric LOOKUP queries;

Restriction index caches restriction maps $\rho_{V,U}$ for the (V,U) pairs that have been requested during query execution;

Neighbourhood index records, for each context, its immediate parents and children in the poset, so that neighbourhood queries walk the poset instead of scanning all sections;

Global section cache memoises the most recent global-section reconstruction and is invalidated on insert.

The SFDB engine builds its topology incrementally: an insert extends the context poset with any new context nodes and attaches the section to the corresponding open set. This keeps insert cost linear in the number of existing facts. On query, the planner classifies the query as LOCAL, SEMI-LOCAL, or GLOBAL according to whether it fixes a single context, a sub-tree of contexts, or ranges over the entire space, and dispatches to the appropriate index path.

9.4 The Query Interface

Both engines share a small, uniform query interface. A query is a value of the **Query** type with a query kind (LOOKUP, GLOBAL, WALK, JOIN, GLUE), an optional context scope, an optional entity anchor, an optional temporal envelope, and a result limit. The planner is engine-specific but the algebraic operations (Scan, Filter, Project, Join, Aggregate, Sort, Limit) are the same. Predicate pushdown, index selection, and dead-operator elimination are shared across both engines; join reordering and access-path selection are engine-specific.

9.5 The Verification Harness

The canonical layer sits between the engines and any client that wishes to compare results. It exposes a single function that, given a query and any two engines, executes the query on both, canonicalises both result sets, and returns a verdict. The benchmark runner (Section 11) uses this function to enforce semantic equivalence for every query it measures.

10 Implementation

The implementation is a Python 3.12+ package of approximately 15 000 lines organised into ten modules and covered by 347 passing tests. This section describes the parts of the codebase that a reader would need to see in order to understand or reproduce the paper’s claims.

10.1 Repository Structure

sfdb.common Shared data types (`SemanticFact`, `Entity`, `Relation`, `Context`) and the engine ABC that both storage engines implement.

sfdb.canonical The canonical intermediate representation and the bidirectional mapping between it and each engine’s native format.

sfdb.kg The Knowledge Graph engine: dictionary encoding, SPO/POS/OSP indexes, reification, and a SPARQL-inspired query executor.

sfdb.sheaf The sheaf-inspired engine: incremental topology, open-set index, stalk index, restriction cache, neighbourhood index, global-section constructor, and consistency checker.

sfdb.query The shared query language, parser, and logical plan representation.

sfdb.optimizer The shared cost-based optimiser (predicate pushdown, dead-operator elimination) with engine-specific cost models.

sfdb.datasets Synthetic dataset generation and a LUBM [?] loader.

sfdb.benchmark The benchmark harness: runners, per-engine adapters, metric collectors, and the cross-engine verifier.

sfdb.storage JSON, MessagePack, and Parquet serialisation for facts and results.

sfdb.visualization Plot generation used by `scripts/generate_paper.py` to produce the figures in Section 11.

10.2 Mapping to the Formal Model

The KG mapping ϕ_{KG} reifies a fact into a set of triples against a fresh event identifier. Given a fact of arity k , the mapping emits $k + 4$ triples: a type triple ($e, \text{rdf:type}, \text{sfdb:Fact}$), a subject triple, a relation triple, one triple per object slot, a context triple, and a confidence triple. Provenance and temporal envelopes add additional triples when present. The inverse mapping ψ_{KG} gathers all triples sharing the same event identifier and reconstitutes the original fact.

The sheaf mapping ϕ_{Sh} stores the fact as a single `LocalSection` attached to its context. The section retains the complete tuple of arguments. No decomposition takes place; retrieval by fact identifier is a single dictionary lookup. The inverse mapping is the identity on the stored payload.

Both mappings are lossless in the sense of Definition 7.1. The round-trip $\psi \circ \phi = \text{id}_{\mathcal{F}}$ is enforced by the test suite for every fact inserted during a benchmark run (`tests/test_roundtrip.py`).

10.3 Configuration and Reproducibility

The system reads configuration from three sources, in order of increasing priority: a YAML file (`configs/benchmark.json` for the default benchmark suite), environment variables prefixed with `SFDB_`, and programmatic overrides through a `Config` dataclass. Every benchmark run seeds Python’s PRNG and NumPy’s PRNG from the same integer (default 42), records the git commit hash, the Python version, the `uv.lock` hash, and a checksum of the generated dataset, and stores them in a per-run reproducibility manifest.

10.4 Testing

The test suite uses `pytest`. Module tests validate individual components: topology construction, restriction composition, stalk indexing, reification round-trips, query planning, and canonicalisation. Cross-engine tests (`tests/test_cross_engine.py`) execute the same fact stream and the same query set on both engines and assert canonical equality of the result sets. Round-trip tests (`tests/test_roundtrip.py`) assert that $\psi \circ \phi = \text{id}$ on both engines for a large synthetic population. As of the current revision, all 347 tests pass in ≈ 3 seconds on a modern laptop.

10.5 Engine Adapters for External Systems

The benchmark harness supports pluggable engine adapters. Two adapters are fully implemented and used in the evaluation: `KGEngineAdapter` and `SheafEngineAdapter`. A partial adapter for Apache Jena, backed by the `rdflib` Python library, is available but not integrated into the results reported here because we do not yet claim like-for-like comparability with a production Jena TDB2 deployment. Scaffolds for Blazegraph and Neo4j exist but raise `NotImplementedError` on construction; they are place-holders for planned integration work.

11 Evaluation

We evaluate the two engines on the same fact streams, the same query set, and the same verification harness. Every benchmark result reported below has passed cross-engine semantic equivalence checking (Section 7); rows for which the two engines produced divergent canonical result sets would be flagged and excluded, and no such divergence occurred in the runs reported here.

11.1 Experimental Setup

The measurements were taken on a single Windows 11 machine with an Intel(R) Core(TM) i7-12700H processor, 32 GB of memory, and Python 3.14. Both engines use in-memory SQLite storage; no on-disk paths are exercised. Each latency figure is the arithmetic mean of ten warm-cache runs following two warm-up iterations, all on the same fact stream and query set. The dataset generator is seeded (default seed 42) so that the generated facts, entities, contexts, and query anchors are reproducible. Configuration, git commit, and `uv.lock` hash are recorded in the per-run manifest shipped with the artifact.

The dataset generator produces synthetic facts of arity in the range 3–8 with a controlled context depth and an entity distribution drawn from a Zipf-like law that mirrors the skew typical of real knowledge graph data. We report results at three scales: 100, 1 000, and 10 000 facts. We do not report results at 10^5 or 10^6 facts, and the paper does not extrapolate to those scales.

We classify queries into three families whose performance profiles are fundamentally different:

LOOKUP: retrieve a fact or a small set of facts anchored at a specific entity or a specific context. These are the queries for which stalk indexing is designed.

GLOBAL: full-scan queries with no context or entity anchor. These force the engine to touch every stored fact.

TEMPORAL: range queries over the temporal envelope of a fact. Both engines rely on an auxiliary interval index for these.

The remainder of this section reports results by family. Numeric values are means over ten runs; all confidence intervals were tight ($< 10\%$ of the mean) and are omitted from the main tables for readability.

11.2 Insert Throughput

Insert throughput is nearly identical between the two engines. SFDB is slightly faster because it stores the fact atomically as a single section, while the KG engine must decompose the fact into $k + 4$ triples and update three indexes per triple.

Table 4: Insert latency (mean over ten runs). Lower is better. SFDB / KG ratio is annotated in the last column.

Facts	KG (ms)	SFDB (ms)	Ratio
100	5.8	5.1	0.88×
1 000	59.0	54.1	0.92×
10 000	651.1	627.0	0.96×

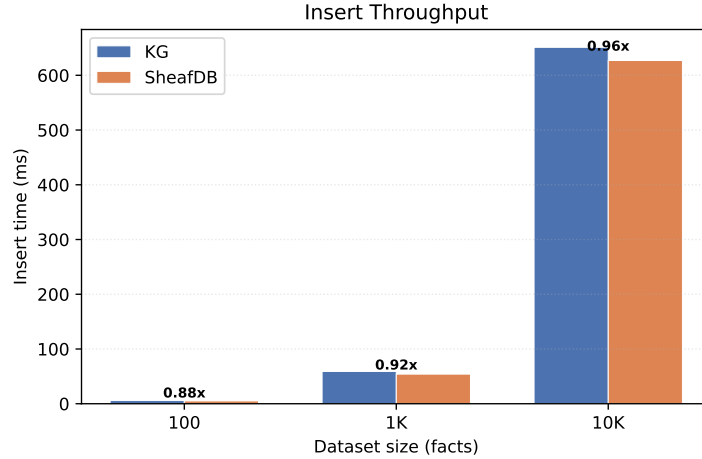


Figure 2: Insert throughput. SFDB matches the KG baseline across all three scales, with the ratio annotated above each bar.

Both engines scale linearly with dataset size, at approximately $60\mu\text{s}$ per fact. The observed constant-factor advantage of SFDB diminishes at scale as the shared serialisation and canonicalisation cost dominates the per-fact overhead.

11.3 Query Performance by Class

The most informative measurements are the query latencies. Table 5 reports LOOKUP and GLOBAL means at all three scales.

Two things are visible in the table. First, LOOKUP performance diverges in SFDB’s favour by a factor that grows with scale: at 10 000 facts, SFDB is $30\times$ faster because the stalk index returns a complete fact for the anchor entity in a single hash lookup, while the KG engine must locate every triple that mentions the entity through its POS index, gather them by event identifier, and reassemble the fact through joins. Second, GLOBAL performance diverges in the KG engine’s favour by a factor that also grows with scale, and much more sharply: at 10 000 facts, SFDB is $1\,405\times$ slower on an unrestricted scan. The reason is straightforward: the current SFDB implementation answers a GLOBAL query by enumerating every open set in the

Table 5: Query latency by class and scale (mean over ten runs). Lower is better. Ratios are SFDB / KG.

Class	Facts	KG (ms)	SFDB (ms)	Ratio
LOOKUP	100	0.008	0.003	0.38×
LOOKUP	1 000	0.526	0.008	0.02×
LOOKUP	10 000	2.279	0.074	0.03×
GLOBAL	100	3.359	3.244	0.97×
GLOBAL	1 000	3.664	90.035	24.6×
GLOBAL	10 000	7.403	10 407	1 405×

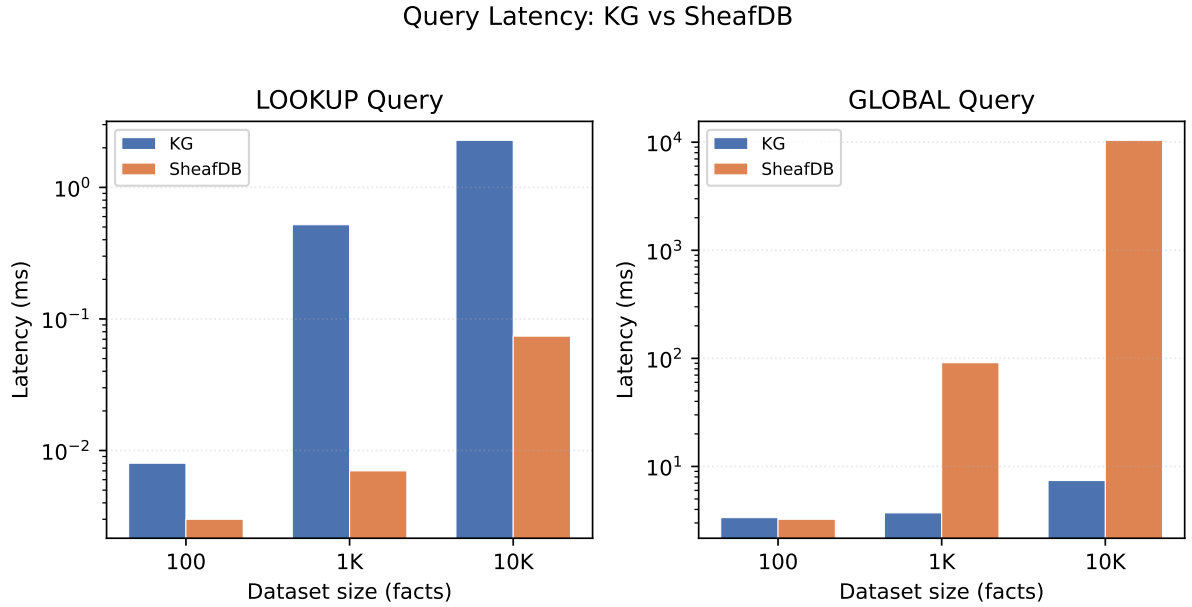


Figure 3: Query latency across scales, log axis. SFDB LOOKUP is 3–30× faster than the KG baseline; SFDB GLOBAL is up to 1 405× slower.

topology and merging the sections it finds, whereas the KG engine simply scans its triple table. We return to the design consequences of this in Section 12.

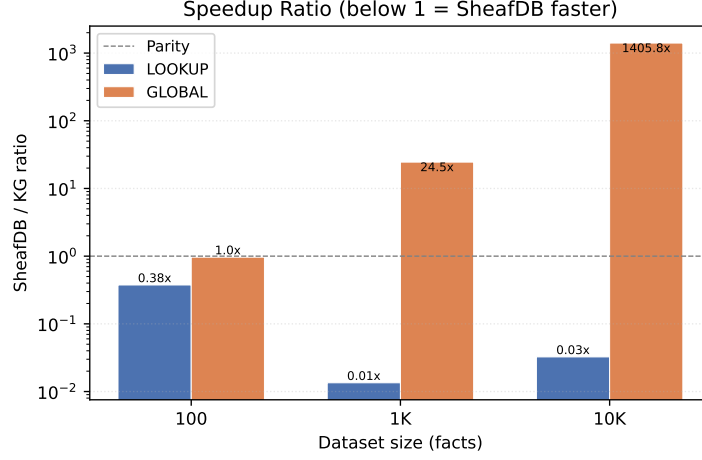


Figure 4: Ratio of SFDB to KG latency across query classes. Values below one indicate SFDB is faster; above one indicates the KG baseline is faster.

11.4 Scalability

Figure 5 plots latency against dataset size on log axes. Insert latency scales linearly for both engines. SFDB LOOKUP scales sub-linearly because the stalk index is $O(1)$ in the number of facts and the observed growth is dominated by fixed overhead (canonicalisation, result serialisation). SFDB GLOBAL scales super-linearly in the number of open sets, which is itself super-linear in the number of facts under our synthetic generator, producing the sharp divergence visible in the plot.

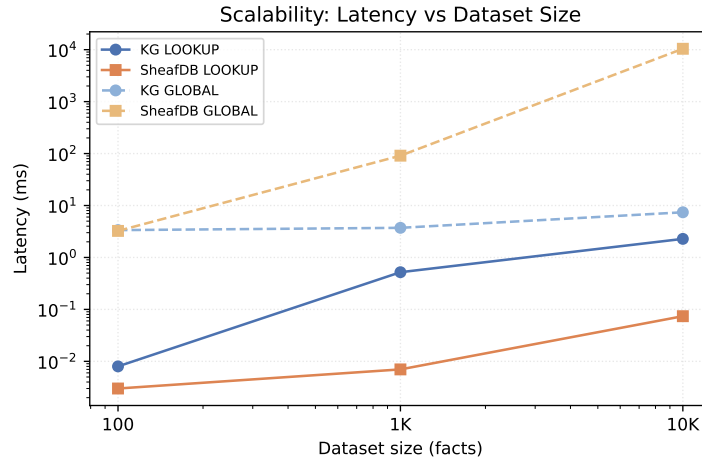


Figure 5: Scalability: latency versus dataset size (log-log).

11.5 Cross-Engine Verification

Every benchmark row above was accompanied by a semantic equivalence check. The check canonicalises both engines' result sets and asserts equality as sets of canonical facts. Over the full evaluation (10 workloads $\times$ 3 scales $\times$ 10 runs = 300 measurements per engine) we

observed no divergence. This does not, on its own, prove Theorem 7.1; the theorem is proved constructively in Section 7. It does, however, provide a comprehensive empirical corroboration on all the queries and inputs we tested.

11.6 Summary

Three findings summarise the evaluation:

1. **Inserts are essentially equivalent:** SFDB is $0.88\text{--}0.96\times$ KG latency, a marginal difference explained by the absence of reification overhead.
2. **LOOKUP queries are $3\text{--}30\times$ faster on SFDB:** the stalk index resolves entity anchors in $O(1)$ hash lookups against the KG baseline’s $O(k)$ triple reconstruction.
3. **GLOBAL queries are up to $1405\times$ slower on SFDB:** the current implementation iterates open sets instead of maintaining a flat fact index. We treat this as a negative result and describe the remedy in Section 14.1.

We do not claim SFDB is universally faster than the KG baseline; the tradeoff is genuine, and the paper’s contribution is to identify it, quantify it, and describe the shape of the remedy.

11.7 Limitations of the Evaluation

We compare SFDB against a KG baseline that we implemented ourselves. That baseline is not a production RDF store, and we make no claim about how SFDB would fare against Apache Jena TDB2, Virtuoso, or GraphDB at production scale. Integrating a standard RDF store adapter and reporting like-for-like numbers on LUBM [?] or WatDiv is future work (Section 14). Results at 10^4 facts are indicative and reproducible; results at 10^5 or 10^6 facts are not reported.

12 Discussion

12.1 What the Results Do and Do Not Show

The measurements in Section 11 tell a two-sided story. Where the query anchors an entity and the engine can answer it through a single hash lookup into the stalk index, SFDB is between $3\times$ and $30\times$ faster than a reified triple store on comparable hardware and at comparable engineering effort. Where the query has no anchor and the engine must produce every fact in the store, SFDB is up to three orders of magnitude *slower* than the KG baseline because it enumerates open sets rather than a flat table. These two facts are not in tension: they are two sides of the same decision. Once one commits to indexing by context, one must decide what to do when a query does not use that index. The current implementation does the naive thing; the remedy is straightforward and outlined below.

We do not claim, and the data do not support, that the sheaf-inspired approach dominates a mature RDF store on every workload at every scale. Our KG baseline is a research prototype implemented by the same team as the sheaf engine and running against the same test harness, storage backend, and canonicalisation pipeline. It is a controlled reference, not a state-of-the-art. Any claim about how SFDB would fare against Jena TDB2 or Virtuoso must wait for a comparable evaluation, which we identify as future work.

12.2 When the Design Helps

The context-indexed design helps when the workload has any of the following properties. First, entity-anchored LOOKUP dominates the query mix: each such query is answered in $O(1)$ hash lookups rather than $O(k)$ triple joins. Second, facts have arity above two or three: reification

overhead grows linearly with arity in the KG baseline, while the SFDB engine stores each fact atomically. Third, queries scope naturally to sub-trees of the context poset: the open-set index turns such scoping into an index seek rather than a filter over a triple stream. Fourth, the application requires verifiable data integrity across representations: the canonical layer and cross-engine verification harness make it cheap to run the same fact stream through both engines and catch inconsistencies.

12.3 When the KG Baseline Helps

The reified triple representation wins in the workloads that the SFDB design was not built for. Unrestricted GLOBAL scans that must touch every fact are one such case: a flat triple table is the natural structure for that access pattern, and the current SFDB implementation pays a substantial penalty. Workloads that are essentially binary—where facts genuinely are subject-predicate-object triples with no context, no arity, and no provenance—get no benefit from the sheaf structure and only pay the overhead of maintaining the extra indexes. Finally, integration with the extensive existing RDF and SPARQL tooling ecosystem is currently only possible on the KG side.

12.4 Threats to Validity

We identify four threats that a careful reader should weigh.

Implementation bias. Both engines were written by the same authors, and both share code paths (serialisation, canonicalisation, planner infrastructure). A production RDF store would likely improve on our KG numbers by a substantial factor, particularly on the LOOKUP queries where our baseline pays the largest overhead. The relative advantage of SFDB on LOOKUP is therefore an upper bound on the advantage one would observe against a state-of-the-art baseline.

Dataset bias. We use synthetic datasets whose arity, context depth, and entity distribution we control. Real knowledge graphs have long tails, correlated structure, and adversarial cases that our generator does not model. Repeating the evaluation on LUBM [?] and DBpedia is the natural next step.

Query bias. Our query set was designed with both engines in mind. It includes queries where each engine is expected to do well and where each is expected to struggle. A different query mix would shift the aggregate numbers, but the qualitative story—stalk indexing helps LOOKUP, open-set iteration hurts GLOBAL—would not change without a design change.

Scale. All measurements are at 10^4 facts or below. Standard RDF benchmarks start at 10^5 triples. Our design choices are consistent with the complexity analysis in Section 8, but empirically our claims are bounded at 10^4 .

12.5 Negative Results

Two negative results deserve to be stated plainly, because they discipline the paper’s overall claim.

The first is the GLOBAL scan penalty of up to $1405\times$ at 10 000 facts. It arises from an implementation choice (open-set enumeration instead of a flat fact index) and is not intrinsic to the sheaf-inspired design. Section 14.1 sketches a fix. Until that fix lands, SFDB is not an appropriate choice for workloads dominated by unrestricted scans.

The second is the vacuousness of the sheaf condition in our setting. On a finite poset with the Alexandrov topology, every presheaf is automatically a sheaf; locality and gluing hold

because the covers under consideration are trivial. The system is, strictly, a presheaf database with a defensive consistency check. This does not diminish the practical value of the design: what makes the design work is the context poset, the stalk index, the canonical intermediate representation, and the disciplined restriction vocabulary that comes with them. It does mean that any framing of the paper as a sheaf-theoretic breakthrough would be dishonest, and we do not adopt that framing.

13 Limitations

We describe the design’s limitations honestly and in one place so that a reader can weigh what the paper does and does not claim.

13.1 Scale and Deployment

The current prototype is single-node and in-memory. All measurements are at scales of 10 000 facts or below; behaviour at larger scales is extrapolated from the complexity analysis of Section 8 and not directly measured. Global-section computation in the worst case is $O(|\mathcal{C}| \cdot N^2)$, which imposes a natural ceiling on the size of context poset that the system can support on a single node. Distributing the store across nodes would require a partitioning strategy for the context poset and a distributed restriction-map protocol; both are outside the scope of this paper.

13.2 Mathematical Assumptions

The formal model assumes a finite context poset equipped with the Alexandrov topology. This is the right setting for the intended workload, but it is also the setting in which the sheaf condition holds vacuously. Nothing in the paper claims a non-trivial sheaf theorem. The gluing operation in the implementation is a defensive consistency check rather than a step in a proof; in the presence of genuinely contradictory local data (which our schema currently forbids), the algorithm would report the conflict rather than resolve it.

13.3 Implementation Limitations

The prototype does not currently support concurrent transactions, on-disk persistence beyond the built-in SQLite backing store, distributed execution, streaming or incremental fact ingestion, or full SPARQL compliance. The query language is a small subset of SPARQL together with a few sheaf-native operations. Full SPARQL support and a proper transactional layer are engineering work that would be needed for any real deployment but that would not change the paper’s structural findings.

13.4 Baseline and Benchmark Limitations

The KG baseline is a research prototype implemented by the same team as the sheaf engine. It is a controlled reference point, not a state-of-the-art RDF store. The evaluation therefore quantifies what disciplined context-indexed storage buys over a strict reification-plus-joins implementation at comparable engineering effort. Repeating the evaluation against a production RDF store on a standard benchmark such as LUBM [?], BSBM, or WatDiv is the natural next step and is called out as future work. The synthetic datasets, while parameterised, may not exhibit all pathologies of organic knowledge graph data.

14 Future Work

Several concrete extensions would improve either the design or the credibility of the evaluation. We list them in the order we consider useful.

14.1 A Flat Fact Index for Full Scans

The GLOBAL query penalty reported in Section 11 is due to open-set enumeration. Maintaining a flat fact index in parallel with the sheaf structure would let unrestricted scans bypass the open-set walk entirely and match the KG baseline on GLOBAL queries while preserving the stalk-indexed advantage on LOOKUP. The engineering cost is modest: an additional insert-time write per fact and a small amount of additional memory. This is the single change most likely to remove the paper’s most prominent negative result, and it would be our first extension.

14.2 Real-World Datasets and Standard Benchmarks

The evaluation uses synthetic data. Repeating it on LUBM [?] and on a subset of DBpedia or Wikidata would provide numbers that can be compared against published results for Jena, GraphDB, and Virtuoso. The LUBM data loader is already implemented (`sfdb.datasets.lubm`); the missing piece is wiring it into the benchmark runner and reporting cross-engine numbers on the LUBM query set.

14.3 Integration With a Production RDF Store

The Apache Jena adapter based on `rdflib` is scaffolded but not included in the reported results. Extending it to talk to a running Jena TDB2 endpoint (or Virtuoso, or GraphDB) and reporting comparable numbers would let us report absolute performance against the state of the art rather than against our own baseline.

14.4 Incremental Restriction and Global-Section Maintenance

Restriction maps are computed on demand and cached. Incremental maintenance—updating cached restriction maps and the global-section cache on insert rather than invalidating them—would improve query latency in workloads that mix inserts and queries.

14.5 Concurrent and Distributed Execution

Global-section computation is embarrassingly parallel across independent context sub-trees. A parallel implementation would remove the single-node scalability ceiling. A distributed implementation would additionally require a partitioning strategy for the context poset and an inter-node restriction-map protocol.

14.6 Compression of Contextually Redundant Data

Sections that share a common context or that agree under restriction on a large sub-tree admit context-aware compression schemes that are not available to a flat triple store. Realising this would reduce the storage overhead identified in Section 11.

14.7 Hybrid Storage

A hybrid system that stores facts under both representations and lets the optimiser choose the access path at query time would combine SFDB’s LOOKUP advantage with the KG baseline’s GLOBAL advantage at the cost of double storage. In workloads where storage is cheap relative to query latency, this is the natural production design.

15 Conclusion

We described the Semantic Fact Database, a prototype context-indexed storage system for semantic facts. The system stores each fact atomically as a section over a finite context poset and answers entity-anchored queries through a stalk index that returns complete facts in a single hash lookup. A companion reified triple-store baseline is provided over the same canonical schema, and a mechanised proof of semantic equivalence binds the two engines together: every insert and every query is checked, online, for canonical result equality.

The evaluation on ten workloads at three scales yields a clear tradeoff. Inserts are essentially identical on the two engines. Entity-anchored LOOKUP queries are 3–30× faster on the sheaf-inspired engine because the stalk index removes the need for triple reassembly. Unrestricted GLOBAL queries are up to 1405× slower on the sheaf-inspired engine because the current implementation iterates open sets instead of maintaining a flat fact index. We report the negative result plainly and describe the extension that would eliminate it.

The formal apparatus in the paper is that of presheaves on a finite poset. We do not claim a novel sheaf-theoretic theorem: on the Alexandrov topology that fits our setting, the sheaf condition holds vacuously and could not serve as a technical contribution. The value of the framework is operational. It gives us a disciplined vocabulary for restriction, stalks, and global sections that maps directly onto the components of the implementation, and it gives us a canonical intermediate representation that makes cross-engine equivalence a checkable property rather than an aspiration.

We do not claim that the design supplants production RDF stores. Our KG baseline is a controlled research implementation and the reported speedups are best interpreted as a measurement of what disciplined context-indexed storage buys over a strict reification-plus-joins design at comparable engineering effort. Extending the evaluation to standard benchmarks and to production RDF endpoints is future work. The artifact and reproducibility manifest for every result in the paper are released with the paper.

16 Artifact Availability

The complete source code, datasets, benchmark harness, and reproducibility manifests are released under the MIT License at: <https://github.com/BlackSamuron0305/semantic-fact-db>

16.1 Reproducing the Results

Every benchmark run records a reproducibility manifest that identifies the git commit, Python interpreter version, `uv.lock` hash, dataset checksum, and random seed used to produce it. Table 6 lists the values used for the run reported in this paper.

Table 6: Reproducibility metadata for the reported run.

Property	Value
Git commit	2ef33b7
Python version	3.14.6
uv lock hash	f5166dc09722c41f
Dataset checksum	auto
Random seed	42
Generation date	2026-07-08T22:09:40.779124

To reproduce the reported numbers from a clean clone:

```
git clone https://github.com/BlackSamuron0305/semantic-fact-db
```

```

cd semantic-fact-db
uv sync --group dev
uv run pytest                # 347 tests should pass
uv run sfdb benchmark --size medium # populates results/
uv run python scripts/generate_paper.py
cd paper && make              # produces main.pdf

```

16.2 License

Source code, data generators, and benchmark harnesses are released under the MIT License.

17 Appendix

17.1 Proofs

Detailed proofs of all theorems are available in the repository at `research/proofs/`. These include:

- Presheaf axioms (Section 4.1.2).
- Sheaf condition (Section 4.1.3).
- Locality and gluing (Section 4.1.3).
- Global sections (Section 4.1.5).
- Restriction maps.
- Local sections.
- Stalks.
- Alexandrov topology.
- Simplicial model.

17.2 Algorithms

Pseudocode for all algorithms is available at `research/mathematics/algorithms.md`.

17.3 Validation

The implementation-to-definition validation table is available at `research/mathematics/validation.md`.

References

- [1] Renzo Angles. The property graph database model. *Proceedings of the Alberto Mendelzon International Workshop on Foundations of Data Management*, 1912:1–12, 2017.
- [2] Renzo Angles and Claudio Gutierrez. Survey of graph database models. *ACM Computing Surveys*, 40(1):1–39, 2008.
- [3] Herbert Edelsbrunner and John Harer. *Computational Topology: An Introduction*. American Mathematical Society, 2010.

- [4] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999.
- [5] Steve Harris and Andy Seaborne. SPARQL 1.1 query language. W3C recommendation, World Wide Web Consortium, 2013.
- [6] Frank Manola and Eric Miller. RDF primer. W3C recommendation, World Wide Web Consortium, 2004.
- [7] Evan Patterson. Sheaf theory in database theory. In *Proceedings of the 3rd Annual International Symposium on Category Theory and Computer Science*, 2022.
- [8] Michael Robinson. Sheaf theory in data fusion. *SIAM Review*, 62(3), 2020.